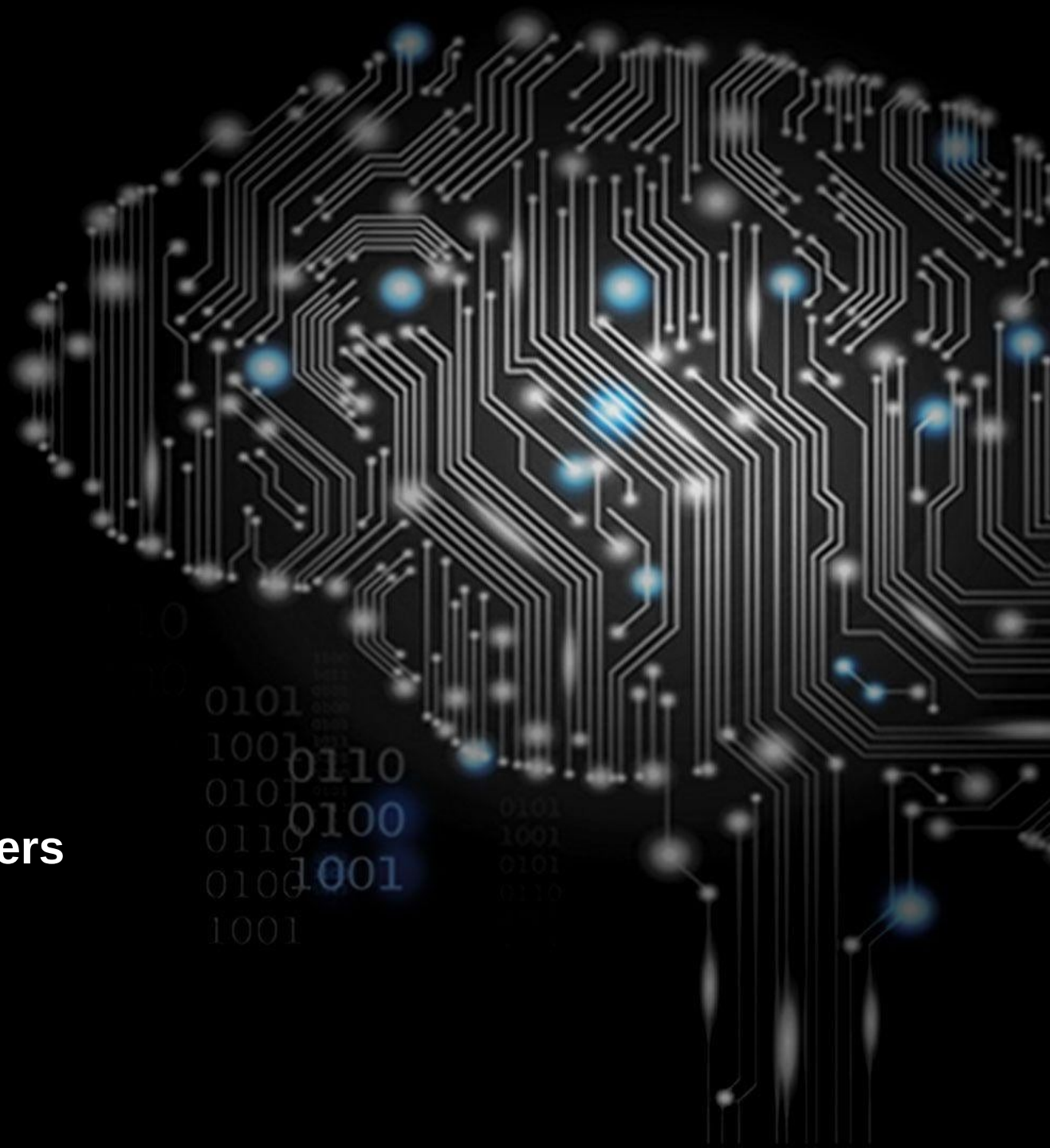


Introduction to Generative AI for Developers

7. Agents

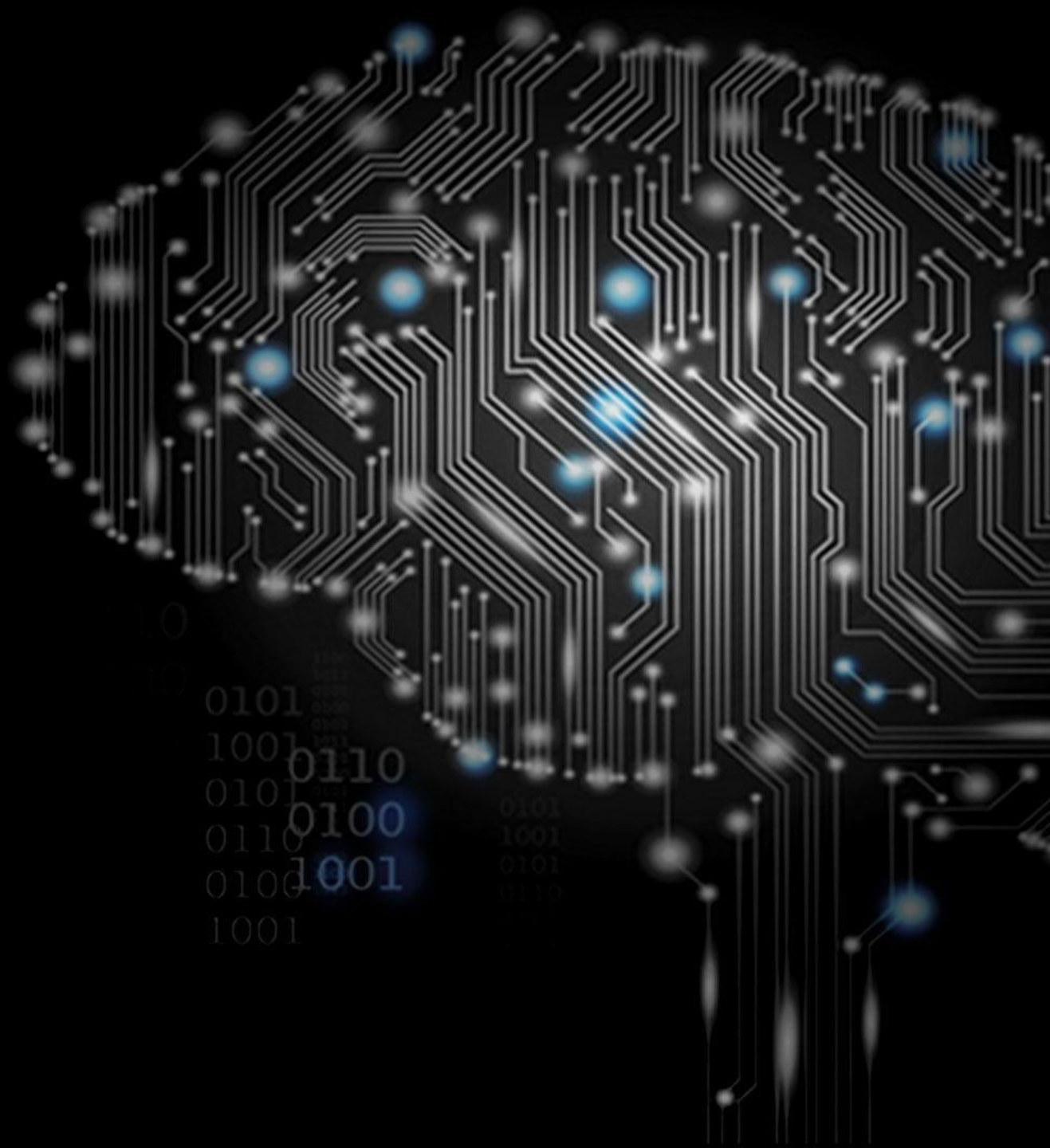
Sections

- Workflows and agents
- Proven composition patterns
- Tool calling
- Memory and state
- Model Context Protocol
- Multi-agent systems
- Safety for agents
- Testing and tracing agents



Agents

1. Workflows and agents



Workflows and agents defined

- Workflow: the application code defines the sequence of steps; the model performs individual steps
  - Predictable paths, bounded cost, straightforward to test and audit
- Agent: the model decides which steps to take and which tools to call, in a loop, until it judges the task complete
  - Flexible for open-ended tasks, but with variable cost and paths that are harder to predict
- Both are built from the same parts: model calls, tools, retrieval and memory
- Anthropic's guidance: find the simplest solution possible, and add complexity only when it demonstrably improves results



When a fixed workflow is safer

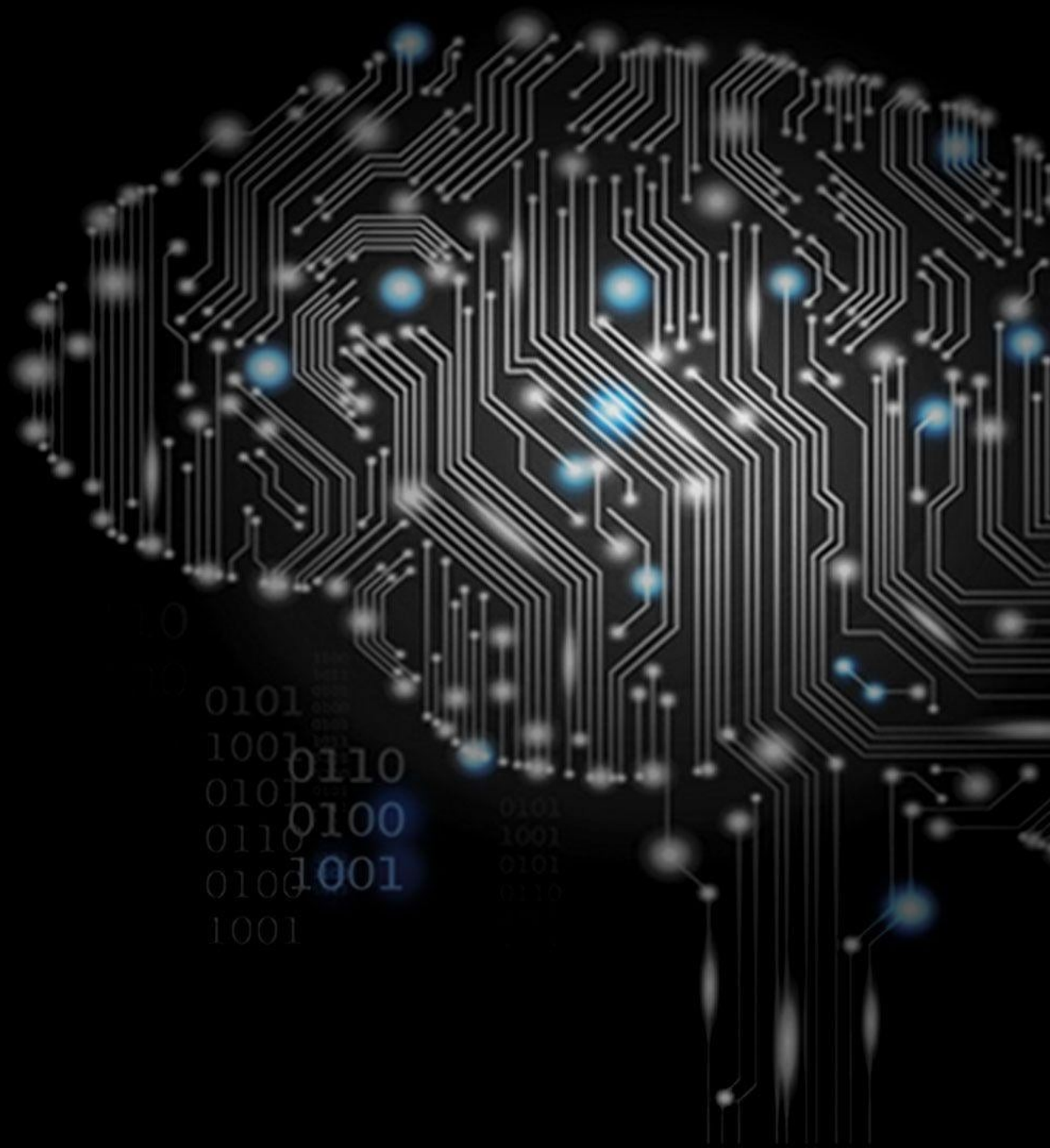
- Prefer a workflow when
  - The steps are known in advance and rarely vary
  - Each step has a clear success check that code can apply
  - Actions have financial, legal or customer impact and must follow a defined procedure
  - Cost and latency must be predictable
  - Auditors need to see that the same procedure was followed every time
- Consider an agent when
  - The number and order of steps cannot be predicted
  - The task needs exploration, such as investigating an unusual account issue across several systems
  - Errors are recoverable, and a human reviews results before they take effect
- Agents carry higher cost and the risk of compounding errors; Anthropic recommends sandboxed testing, guardrails and stopping conditions such as a maximum number of iterations



A spectrum of autonomy

- Single model call with structured output (Units 2 to 4)
- Fixed workflow of several model calls (Section 2 patterns)
- Workflow with a bounded tool-using step: the model may call a small set of read-only tools
- Agent with read tools and proposed write actions that a human approves
- Agent that executes write actions within strict limits
- Move along the spectrum only when evaluation shows the extra autonomy improves outcomes, and add controls at each step





Agents

2. Proven composition patterns

Prompt chaining

- Split a task into a fixed sequence of model calls, each consuming the previous output
- Add programmatic checks between steps, so a failure stops the chain early
- Trades extra latency for higher accuracy on each step
- Example: extract facts from a service request, then check them against policy, then draft a resolution

```
ServiceFacts facts = chatClient.prompt()
        .user(u -> u.text(extractTemplate).param("request", request))
        .call()
        .entity(ServiceFacts.class);

if (facts.accountId() == null) {
    return Resolution.needsHuman("No account identified");   // gate between steps
}

PolicyCheck check = chatClient.prompt()
        .user(u -> u.text(policyTemplate).param("facts", facts.toString()))
        .call()
        .entity(PolicyCheck.class);

return chatClient.prompt()
        .user(u -> u.text(draftTemplate).param("facts", facts.toString()).param("policy", check.toString()))
        .call()
        .entity(Resolution.class);
```



Routing

- Classify the input first, then send it to a specialized prompt, model or workflow
- Lets each route be optimized and tested separately
- Can send simple requests to a small, low-cost model and hard ones to a stronger model
- The triage service from Day 1 is already a router

```
public enum RequestType { ADDRESS_CHANGE, FEE_DISPUTE, CARD_REPLACEMENT, OTHER }

Resolution handle(String request) {
    RequestType type = routerClient.prompt().user(request).call().entity(RequestType.class);
    return switch (type) {
        case ADDRESS_CHANGE   -> addressWorkflow.run(request);
        case FEE_DISPUTE      -> feeDisputeWorkflow.run(request);
        case CARD_REPLACEMENT -> cardWorkflow.run(request);
        case OTHER            -> Resolution.needsHuman("Unrecognized request type");
    };
}
```



Parallelization

- Run independent model calls at the same time and combine the results
  - Sectioning: different subtasks in parallel, such as separate checks for policy, tone and PII
  - Voting: the same task several times, combining answers for higher confidence
- Reduces wall-clock latency; total token cost is unchanged or higher
- Virtual threads make this straightforward in Java 21 and later

```
List<Check> checks = List.of(policyCheck, toneCheck, piiCheck);

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<CompletableFuture<CheckResult>> futures = checks.stream()
            .map(c -> CompletableFuture.supplyAsync(() -> c.apply(draft), executor))
            .toList();
    List<CheckResult> results = futures.stream().map(CompletableFuture::join).toList();
    boolean allPass = results.stream().allMatch(CheckResult::pass);
}
```



Orchestrator-workers

- An orchestrator model breaks a task into subtasks at runtime, delegates them to worker calls, and combines the results
- Differs from parallelization because the subtasks are not known in advance
- Suits tasks such as reviewing a complex complaint that touches several products, where the needed checks depend on the content
- Constrain the orchestrator's output with a schema: a bounded list of subtasks from an allowed set

```
public record Plan(List<Subtask> subtasks) {}
public record Subtask(WorkerType worker, String instruction) {}
public enum WorkerType { ACCOUNT_HISTORY, POLICY_LOOKUP, CARD_STATUS }

Plan plan = orchestrator.prompt().user(complaint).call().entity(Plan.class);
List<String> findings = plan.subtasks().stream()
        .limit(5)                                          // bound the fan-out
        .map(s -> workers.get(s.worker()).run(s.instruction()))
        .toList();
String summary = synthesizer.prompt()
        .user(u -> u.text(synthesisTemplate).param("findings", String.join("\n", findings)))
        .call()
        .content();
```

Evaluator-optimizer

- One call generates a draft; another evaluates it against explicit criteria; the draft is revised until it passes
- Useful when clear evaluation criteria exist and revision measurably improves results
- Always bound the loop with a maximum number of iterations, and fall back to a human when it is reacheded set

```java
public record Verdict(boolean pass, String feedback) {}

Resolution draftWithReview(String request) {
    Resolution draft = generator.prompt().user(request).call().entity(Resolution.class);
    for (int i = 0; i < 3; i++) {
        final String current = draft.text();
        Verdict v = evaluator.prompt()
                .user(u -> u.text(rubricTemplate).param("draft", current))
                .call()
                .entity(Verdict.class);
        if (v.pass()) return draft;
        draft = generator.prompt()
                .user(u -> u.text(reviseTemplate).param("draft", current).param("feedback", v.feedback()))
                .call()
                .entity(Resolution.class);
    }
    return Resolution.needsHuman("Draft did not pass review after 3 revisions");
}
```



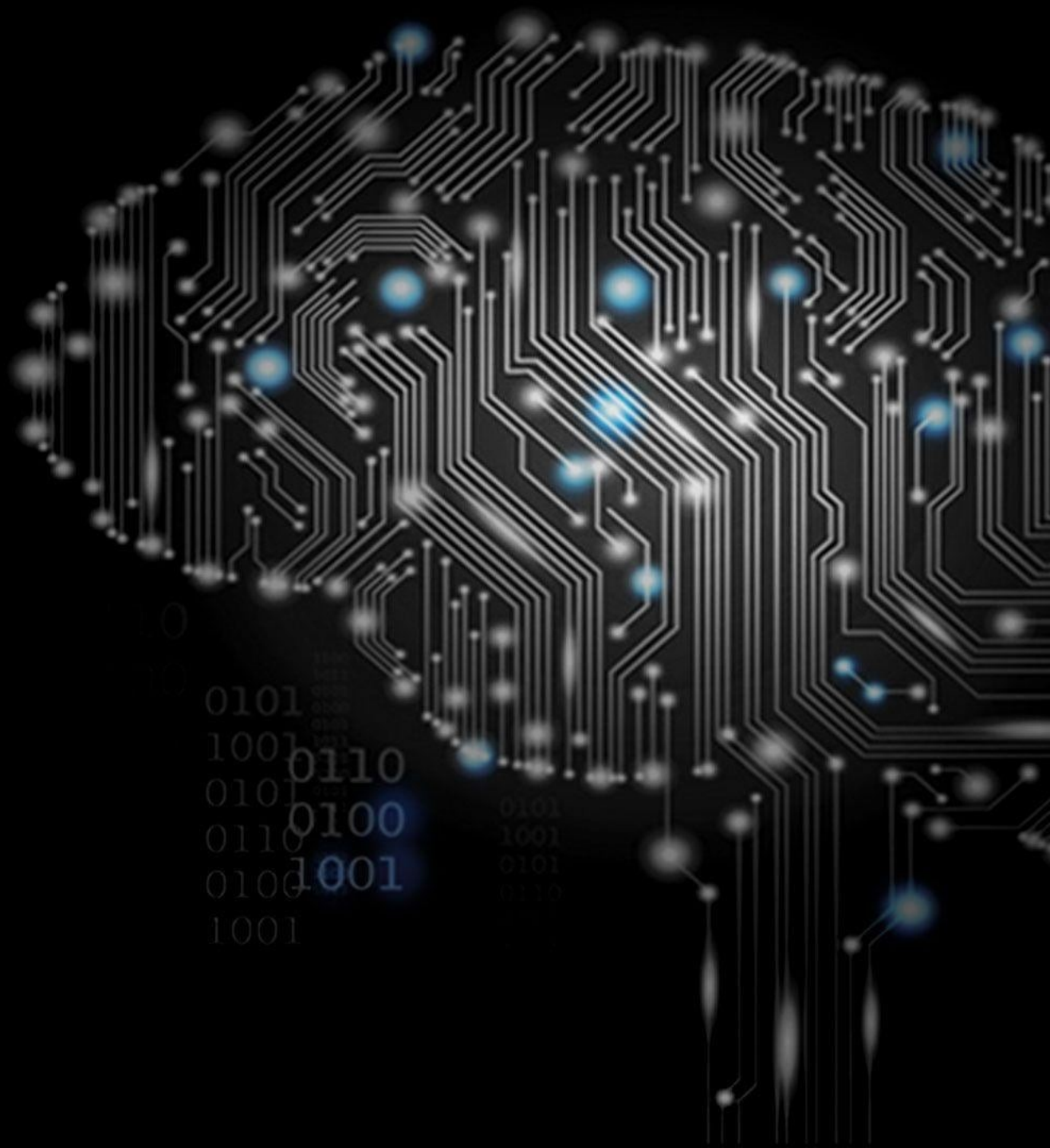
Combining patterns

- Real systems combine patterns
  - Route the request, chain the steps for that route, parallelize the checks, and use an evaluator before release
- Each pattern adds model calls; measure the cost and latency of the whole composition
- Keep the composition in Java code, where it is visible, testable and reviewable
- Introduce an autonomous agent only for the parts that genuinely need it



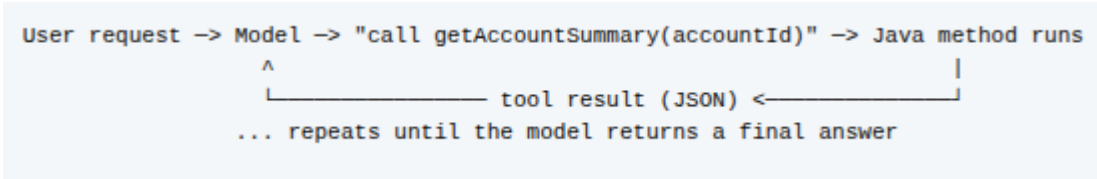
Agents

3. Tool calling



How tool calling works

- The application sends the model a list of tools, each with a name, description and parameter schema
- The model may respond with a request to call one or more tools, with arguments
- The application executes the tools and sends the results back
- The model continues, calling more tools or producing a final answer
- The model never executes anything itself; your code decides what each tool does and whether to run it



Defining tools with @Tool

- Annotate methods with @Tool; Spring AI generates the tool definition and schema from the method signature
- Describe the tool and each parameter; the description is how the model decides when to use the tool
- Register tools on a ChatClient per request with tools(...), or as defaults with defaultTools(...)



Defining tools with @Tool

```
@Component
class AccountReadTools {

    private final AccountService accounts;

    AccountReadTools(AccountService accounts) {
        this.accounts = accounts;
    }

    @Tool(description = """
            Get a summary of the customer's account: type, status, open date and current fees.
            Use this before answering any question about the customer's account.""")
    AccountSummary getAccountSummary(ToolContext context) {
        String accountId = (String) context.getContext().get("accountId");
        return accounts.summary(accountId);
    }

    @Tool(description = "List the customer's transactions in a date range, newest first, at most 50.")
    List<TransactionView> listTransactions(
            @ToolParam(description = "Start date, ISO-8601, e.g. 2026-09-01") LocalDate from,
            @ToolParam(description = "End date, ISO-8601, inclusive") LocalDate to,
            ToolContext context) {
        return accounts.transactions((String) context.getContext().get("accountId"), from, to, 50);
    }
}
```



Designing tools the model can use well

- Treat tool design as interface design for a new user
  - Anthropic recommends writing tool descriptions as you would a docstring for a junior developer, with examples, edge cases and boundaries between similar tools
- Make mistakes hard to make ("poka-yoke")
  - Use enums and typed parameters, explicit date formats, bounded result sizes
- Keep the tool set small and distinct; overlapping tools confuse the model
- Return results the model can read easily: concise JSON with clear field names and units
- Test tools with many example requests and refine descriptions where the model misuses them



The tool-calling loop in Spring AI 2.0

- ChatClient auto-registers a ToolCallingAdvisor that runs the loop
  - Send the request with tool definitions; execute any requested tools; append the results; repeat until the model returns an answer without tool calls
- Exactly one tool-calling advisor is allowed in a ChatClient's chain
- The loop runs inside the advisor chain, so other advisors (memory, guardrails, logging) see each round
- For full manual control, automatic tool execution can be disabled per call or globally, and the application drives the loop itself
- Provider note: tool calling with OpenAI's newest models depends on the API Spring AI uses
  - Spring AI 2.0.2 uses OpenAI's Chat Completions API; OpenAI requires the Responses API for tool calling on GPT-6 Astra; Spring AI 2.1 (in preview) adds Responses API support

```
Resolution resolution = agentClient.prompt()
        .user(serviceRequest)
        .tools(accountReadTools, policyTools, actionProposalTools)
        .toolContext(Map.of("accountId", session.accountId(), "agentUser", session.staffId()))
        .call()
        .entity(Resolution.class);
```



Passing identity and context safely

- ToolContext carries application data to tools without exposing it to the model
  - Customer and account identifiers, the staff user's identity, tenant, correlation ID
- Never let the model choose whose data to access
  - A tool that accepts an account number from the model can be steered by prompt injection to read another customer's account
- Tools enforce authorization using the identity from ToolContext, exactly as a service method would for an API call
- Default and per-request tool contexts are merged, with per-request values taking precedence

```
@Tool(description = "Get the customer's current card status")
CardStatus getCardStatus(ToolContext context) {
    var accountId = (String) context.getContext().get("accountId");
    var staffId = (String) context.getContext().get("agentUser");
    authorization.requireCanView(staffId, accountId);     // same check as
    return cards.status(accountId);
}
```



Iteration and cost limits

- An agent loop can run longer and cost more than expected; set limits on every dimension
- Spring AI tool call limits, per turn
  - Maximum calls per tool (default 40) and in total (default 150)
  - On breach, either stop with a defined finish reason or return an error to the model
- Token and cost budgets: track usage across the loop (usage is aggregated in the response) and stop when a budget is reached
- Wall-clock timeouts for the whole agent run
- The defaults are generous; set much lower limits for a back-office agent

```yaml
spring:
  ai:
    tools:
      limits:
        max-calls-per-tool-default: 5
        max-calls-per-tool:
          listTransactions: 3
        max-total-tool-calls: 15
        on-limit-exceeded: THROW      # loop stops; the response carries the limit finish reason
```



Tool errors

- By default, a runtime exception in a tool is sent back to the model as a message, so it can recover or explain
  - Checked exceptions and errors are always rethrown
- Choose deliberately
  - Recoverable conditions (no transactions in range): return a clear result or message the model can act on
  - Security or integrity failures (authorization denied): throw, stop the run and alert
- Avoid leaking internal details in error messages sent to the model
- spring.ai.tools.throw-exception-on-error=true makes all tool errors propagate to the caller

```
@Bean
ToolExecutionExceptionProcessor toolErrors() {
    return ex -> {
        if (ex.getCause() instanceof AccessDeniedException) {
            throw ex;                                          // stop the agent run
        }
        return "The tool failed: " + safeSummary(ex);          // returned to the model
    };
}
```



Tool errors

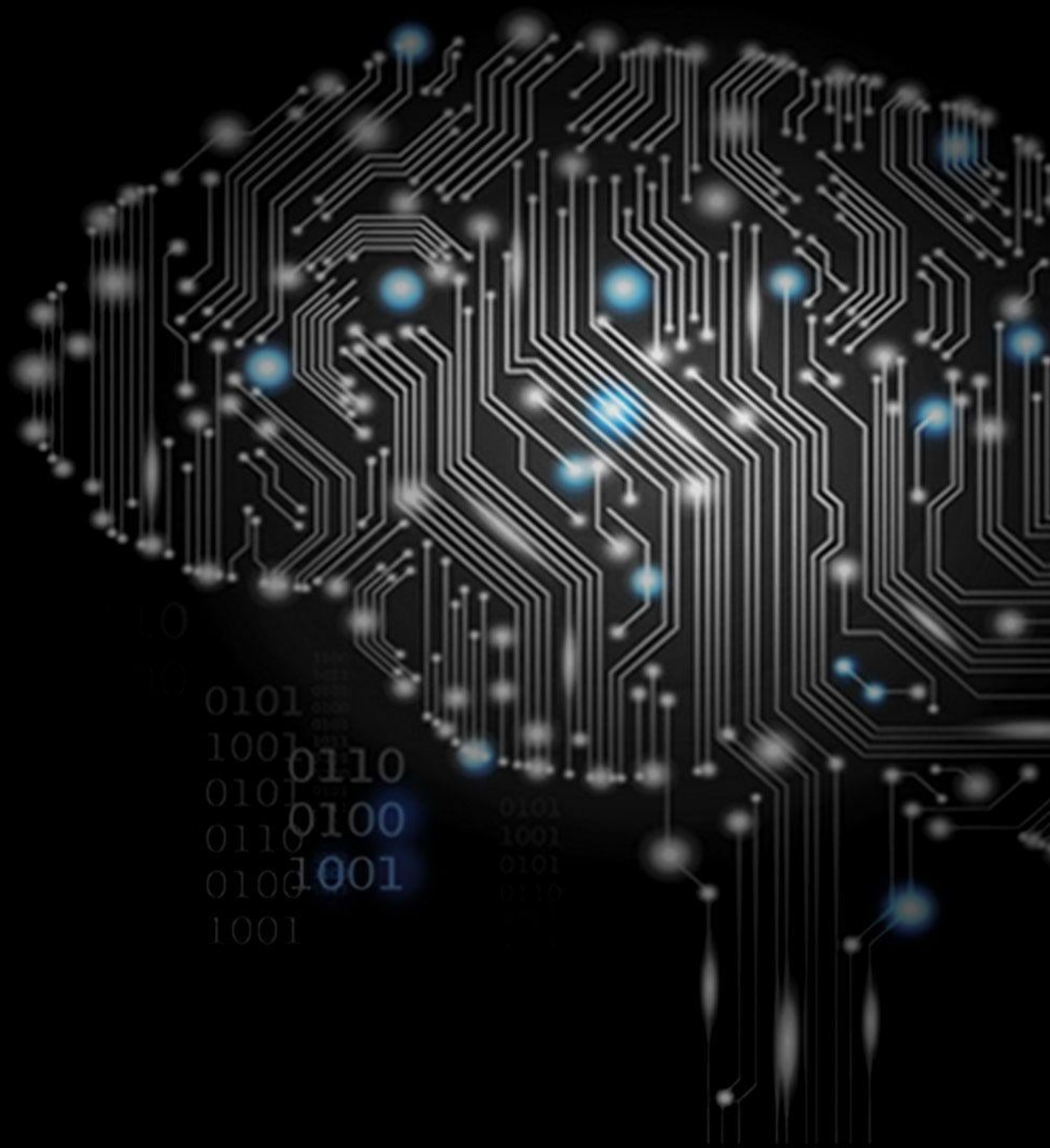
- By default, a runtime exception in a tool is sent back to the model as a message, so it can recover or explain
  - Checked exceptions and errors are always rethrown
- Choose deliberately
  - Recoverable conditions (no transactions in range): return a clear result or message the model can act on
  - Security or integrity failures (authorization denied): throw, stop the run and alert
- Avoid leaking internal details in error messages sent to the model
- spring.ai.tools.throw-exception-on-error=true makes all tool errors propagate to the caller

```
@Bean
ToolExecutionExceptionProcessor toolErrors() {
    return ex -> {
        if (ex.getCause() instanceof AccessDeniedException) {
            throw ex;                                          // stop the agent run
        }
        return "The tool failed: " + safeSummary(ex);          // returned to the model
    };
}
```



Agents

4. Memory and state



Conversation memory

- Models are stateless; memory resends relevant earlier messages on each call
- Spring AI's ChatMemory decides what to keep; a repository stores it
  - The default MessageWindowChatMemory keeps the most recent messages (20 by default) and always keeps system messages
- MessageChatMemoryAdvisor adds memory to a ChatClient
  - A conversation ID must be supplied on every call; there is no default, and omitting it is an error
- Derive conversation IDs on the server, never from client input alone

```
@Bean
ChatClient agentClient(ChatClient.Builder builder, ChatMemory chatMemory) {
    return builder
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
            .build();
}

String conversationId = staffUser.id() + ":" + serviceRequest.id();
agentClient.prompt()
        .user(followUpQuestion)
        .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
        .call()
        .content();
```



Persistent state

- Persist memory across restarts and instances with a repository
  - JDBC (PostgreSQL, MySQL and MariaDB, SQL Server, HSQLDB, Oracle), Cassandra, MongoDB, Neo4j, Redis
- Limitation: the JDBC, Cassandra and MongoDB repositories do not store tool-call messages
  - Use a session store designed for tool-calling histories (the Spring AI Session project) if full agent turns must persist
- Keep business state in systems of record; model memory is only context for the model
  - Case status, pending approvals and completed actions belong in the bank's databases and workflow systems
  - The agent reads current state through tools on each run

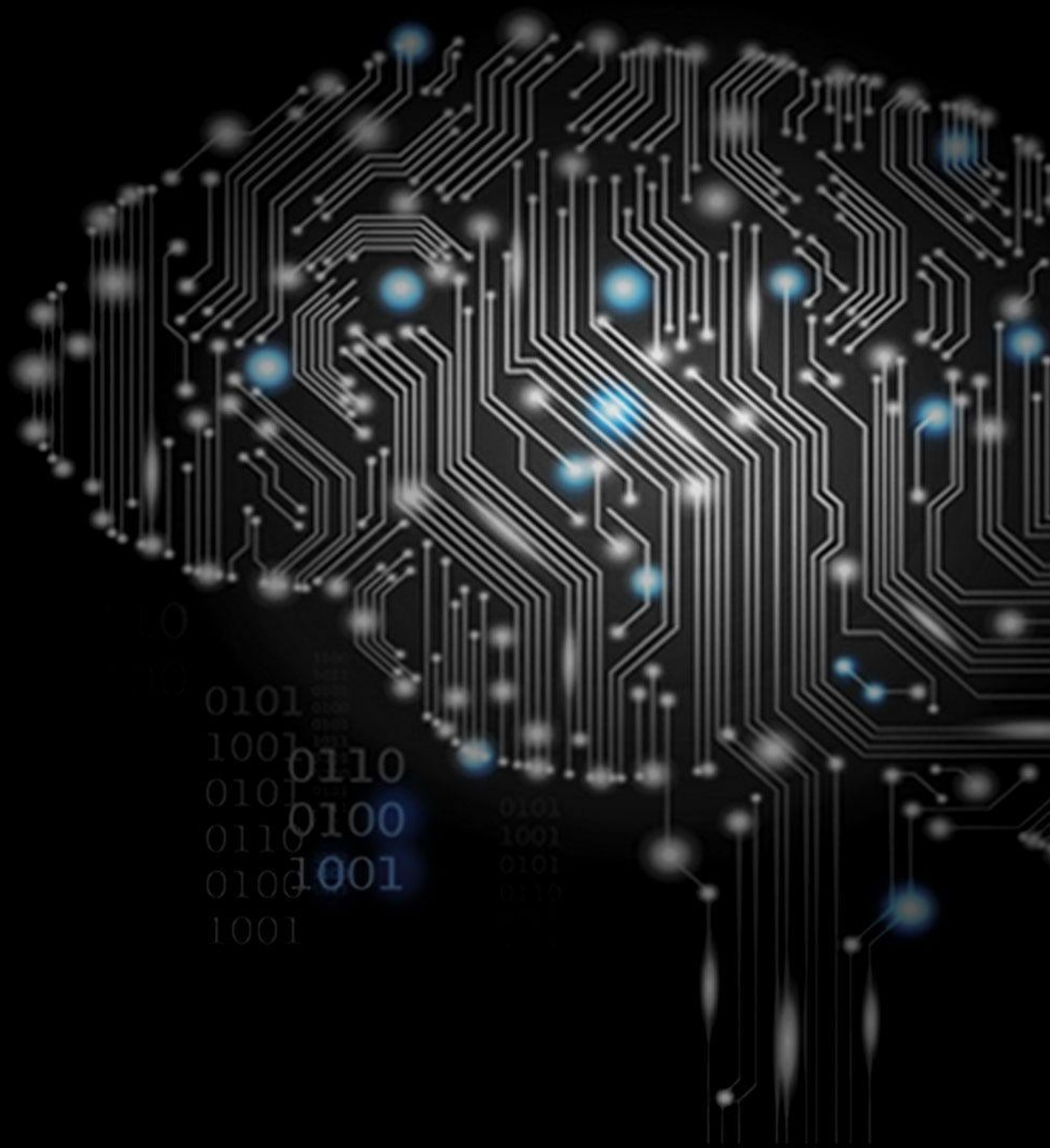
```
@Bean
ChatMemory chatMemory(JdbcChatMemoryRepository repository) {
    return MessageWindowChatMemory.builder()
            .chatMemoryRepository(repository)
            .maxMessages(20)
            .build();
}
```



Memory risks

- Memory poisoning: injected instructions stored in memory influence later turns or other tasks
  - OWASP lists Memory and Context Poisoning (ASI06) among agentic risks
- Personal data retention: memory stores customer details; apply retention, masking and access controls (Unit 6)
- Context growth: long memories raise cost and can dilute instructions
- Cross-contamination: a wrong or shared conversation ID mixes customers' information
- Mitigations: scoped conversation IDs, window limits, masking before storage, expiry, and treating remembered content as data

Memory poisoning: injected instructions stored in memory influence later turns or other tasks
OWASP lists Memory and Context Poisoning (ASI06) among agentic risks
Personal data retention: memory stores customer details; apply retention, masking and access controls (Unit 6)
Context growth: long memories raise cost and can dilute instructions
Cross-contamination: a wrong or shared conversation ID mixes customers' information
Mitigations: scoped conversation IDs, window limits, masking before storage, expiry, and treating remembered content as data



Agents

5. Model Context Protocol

What MCP is

- The Model Context Protocol is an open protocol for connecting AI applications to tools and data
  - An MCP server exposes tools, resources and prompts; an MCP client in the AI application discovers and uses them
- Transports: standard input and output for local processes; streamable HTTP for network services
- Benefits
  - One integration serves many AI clients; tools are described once and discovered at runtime
  - A standard boundary between the agent and the systems it uses
- Spring AI 2.0 supports both sides through the MCP Java SDK 2.0 and Spring Boot starters



Exposing an existing Spring service as an MCP server

- Add the MCP server starter and annotate methods with @McpTool and @McpToolParam
- Delegate to the existing service layer, so business rules and authorization are reused
- Choose the transport: streamable HTTP for a network service
- The endpoint is POST /mcp by default



Exposing an existing Spring service as an MCP server

```xml
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
</dependency>
```

```yaml
spring:
  ai:
    mcp:
      server:
        protocol: STREAMABLE
```

```java
@Component
class AccountServicingMcpTools {

    private final AccountService accounts;

    AccountServicingMcpTools(AccountService accounts) {
        this.accounts = accounts;
    }

    @McpTool(name = "get_account_summary",
             description = "Get type, status, open date and current fees for an account the caller may view.")
    AccountSummary getAccountSummary(
            @McpToolParam(description = "Internal account reference", required = true) String accountRef) {
        return accounts.summaryForCurrentCaller(accountRef);   // authorization inside the service
    }
}
```



Securing an MCP server

- Spring AI's HTTP MCP transports are unauthenticated by default
  - Any client that can reach the endpoint can list and call every tool
- Before exposing a server beyond localhost
  - Put Spring Security (or the MCP security project) in front of the endpoint
  - Authenticate clients, and authorize each tool call against the caller's identity
- Expose only the tools that are needed, with read and write tools separated
- Log every tool call with the caller identity for audit
- Treat MCP servers as production APIs: change management, versioning, monitoring



Consuming MCP servers from an agent

- Add the MCP client starter and configure connections; Spring AI discovers each server's tools
- Discovered tools are provided as a ToolCallbackProvider, which can be registered on a ChatClient like local tools
- Filter which tools are exposed to the model, and control name prefixes when servers have tools with the same name



Consuming MCP servers from an agent

```yaml
spring:
  ai:
    mcp:
      client:
        request-timeout: 10s
        streamable-http:
          connections:
            account-servicing:
              url: http://localhost:8081
```

```java
@Bean
ChatClient agentClient(ChatClient.Builder builder, SyncMcpToolCallbackProvider mcpTools) {
    return builder
            .defaultToolCallbacks(mcpTools)
            .build();
}

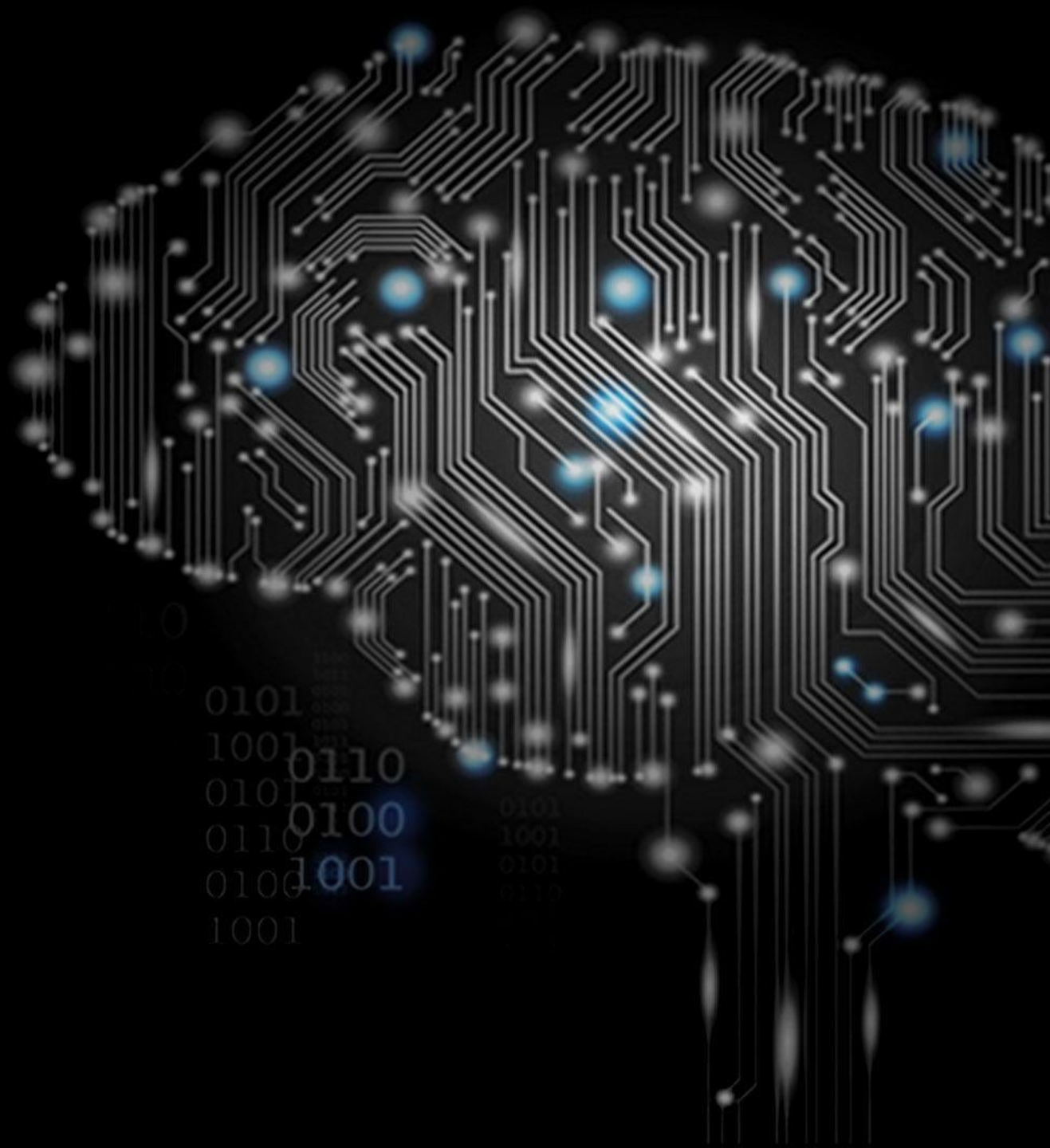
@Component
class ReadOnlyToolFilter implements McpToolFilter {
    @Override
    public boolean test(McpConnectionInfo connection, McpSchema.Tool tool) {
        return tool.name().startsWith("get_") || tool.name().startsWith("list_");
    }
}
```



MCP risks

- Untrusted servers: third-party MCP servers can return manipulated results or misleading tool descriptions
  - OWASP lists Agentic Supply Chain Vulnerabilities (ASI04)
- Tool results as an injection channel: text returned by a tool is data, and must not be treated as instructions
- Over-broad exposure: an MCP server that offers write tools to every client
- Shared sessions and identities in multi-user applications
- Mitigations: approved server list, pinned versions, tool filters, authentication, tool-result delimiting and output guardrails





Agents

6. Multi-agent systems

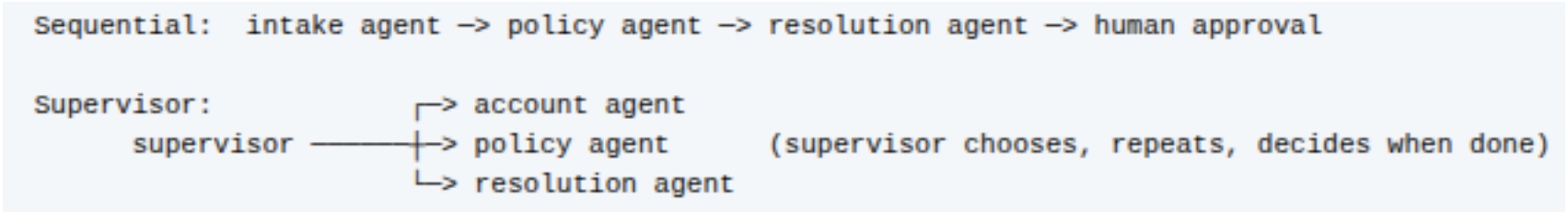
When to use more than one agent

- Separate agents can be justified when parts of a task need
  - Different tools and permissions, such as a read-only research agent and an action agent
  - Different instructions or models, such as a policy specialist grounded in documents
  - Separate context, so one long task does not overload a single context window
- Costs
  - More model calls, more latency, more places for errors to compound
  - OWASP lists Insecure Inter-Agent Communication (ASI07) and Cascading Failures (ASI08)
- Start with one agent and well-designed tools; split only when evaluation shows a benefit



Sequential and supervisor designs

- Sequential: agents run in a fixed order, each passing results to the next
  - Predictable and easy to test; essentially prompt chaining with agents as steps
- Supervisor: a planning agent decides which specialist agent to call next, and when the task is complete
  - Flexible; harder to predict and test; needs iteration limits
- Shared state: agents read and write a common context, such as a case record
- Choose sequential unless the order of specialist steps genuinely varies



Specialist agents as tools in Spring AI

- A simple way to build a supervisor in Spring AI: wrap each specialist agent's ChatClient in a @Tool method
- The supervisor sees each specialist as a tool with a clear description and decides when to call it
- Each specialist keeps its own system prompt, tools, model and guardrails



Specialist agents as tools in Spring AI

```java
@Component
class SpecialistAgents {

    private final ChatClient policyAgent;     // grounded in policy documents, read-only
    private final ChatClient accountAgent;    // has account read tools

    @Tool(description = "Ask the policy specialist a question about servicing policy. Returns an answer with document citations.")
    String askPolicySpecialist(@ToolParam(description = "A specific policy question") String question) {
        return policyAgent.prompt().user(question).call().content();
    }

    @Tool(description = "Ask the account specialist for facts about the current customer's account and transactions.")
    String askAccountSpecialist(@ToolParam(description = "A specific factual question") String question,
                                ToolContext context) {
        return accountAgent.prompt().user(question).toolContext(context.getContext()).call().content();
    }
}
```



LangChain4j's agentic module

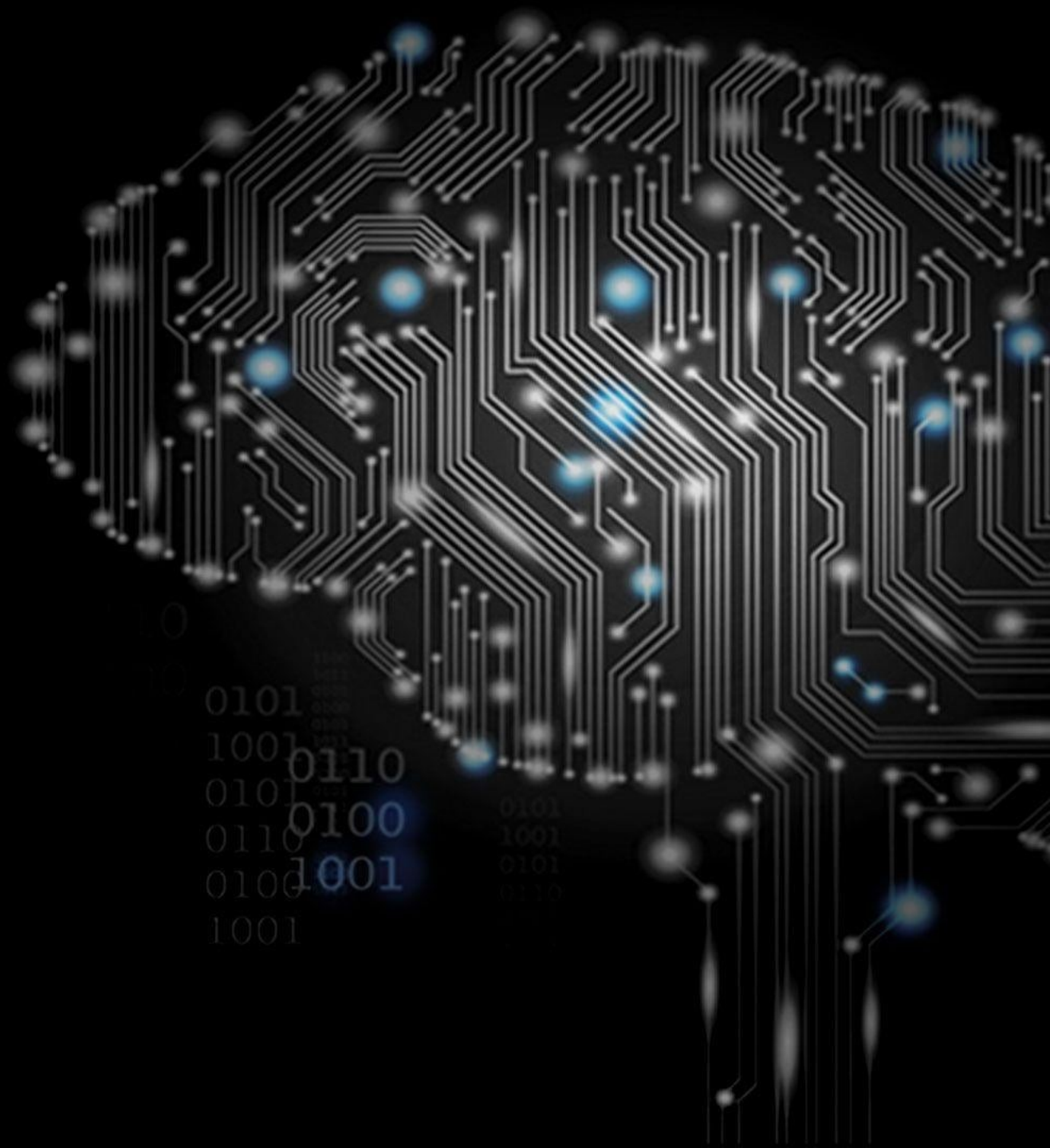
- LangChain4j provides agent building blocks: agents declared with @Agent, and builders for composition
  - sequenceBuilder, parallelBuilder, loopBuilder (with a maximum number of iterations), conditionalBuilder and supervisorBuilder
- Agents share data through an AgenticScope; each agent writes its result under an output key
- The module is documented as experimental and subject to change

```
var intake = AgenticServices.agentBuilder(IntakeAgent.class)
        .chatModel(model).outputKey("facts").build();
var policy = AgenticServices.agentBuilder(PolicyAgent.class)
        .chatModel(model).outputKey("policyCheck").build();
var resolver = AgenticServices.agentBuilder(ResolutionAgent.class)
        .chatModel(model).outputKey("resolution").build();

UntypedAgent pipeline = AgenticServices.sequenceBuilder()
        .subAgents(intake, policy, resolver)
        .outputKey("resolution")
        .build();

String resolution = (String) pipeline.invoke(Map.of("request", serviceRequest));
```





Agents

7. Safety for agents

Least-privilege tools

- Give each agent only the tools its task needs, and give each tool only the access it needs
  - Separate read tools from write tools, in separate classes and, ideally, separate MCP servers
  - Back each tool with a service account or delegated credential scoped to that tool's function
- Narrow parameters: specific operations ("reverse fee F-123") instead of general ones ("update account")
- Enforce authorization in the tool, using the identity from ToolContext
- Review the tool set like a permission grant: who approved it, and what is the worst a misused tool could do?



Read versus write, and human approval

- Read actions: the agent may call them freely within limits
- Write actions: the agent proposes; a person approves; the application executes
  - The write tool records a pending action and returns its ID; it changes nothing in the account
  - A staff member reviews the proposal with the agent's reasoning and evidence, then approves or rejects
  - Approved actions are executed by ordinary application code, outside the model loop
- The model therefore never holds the ability to change customer accounts directly

```
@Component
class ActionProposalTools {

    private final PendingActionRepository pending;

    @Tool(description = """
            Propose reversing a specific fee. This does not reverse the fee;
            it creates a request that a staff member must approve.""")
    String proposeFeeReversal(
            @ToolParam(description = "Fee identifier from listTransactions, e.g. F-123") String feeI
            @ToolParam(description = "Reason citing the policy document ID") String reason,
            ToolContext context) {
        var action = PendingAction.feeReversal(
                (String) context.getContext().get("accountId"), feeId, reason,
                (String) context.getContext().get("agentUser"));
        pending.save(action);
        return "Proposal " + action.id() + " created and awaiting staff approval.";
    }
}
```

Guardrails on tool inputs and outputs

- Tool inputs: validate every argument the model supplies, as you would validate API input
  - Types, ranges, formats, ownership (does this fee belong to this account?), business rules
- Tool outputs: treat results as untrusted data before they return to the model
  - Mask personal data the model does not need; truncate large results; strip or delimit text that could contain instructions
- Apply the Unit 5 guardrails to the agent's final output as well
- Log every tool call, its validated arguments and the result size, for audit



Guardrails on tool inputs and outputs

```java
@Tool(description = "List the customer's transactions in a date range, newest first, at most 50.")
List<TransactionView> listTransactions(LocalDate from, LocalDate to, ToolContext context) {
    if (from.isAfter(to) || from.isBefore(LocalDate.now().minusYears(2))) {
        throw new IllegalArgumentException("Date range must be within the last two years, start before end");
    }
    return accounts.transactions(accountId(context), from, to, 50).stream()
            .map(t -> t.withDescription(piiMasker.mask(t.description())))   // merchant text is untrusted
            .toList();
}
```



The OWASP agentic risks applied to Lab 4

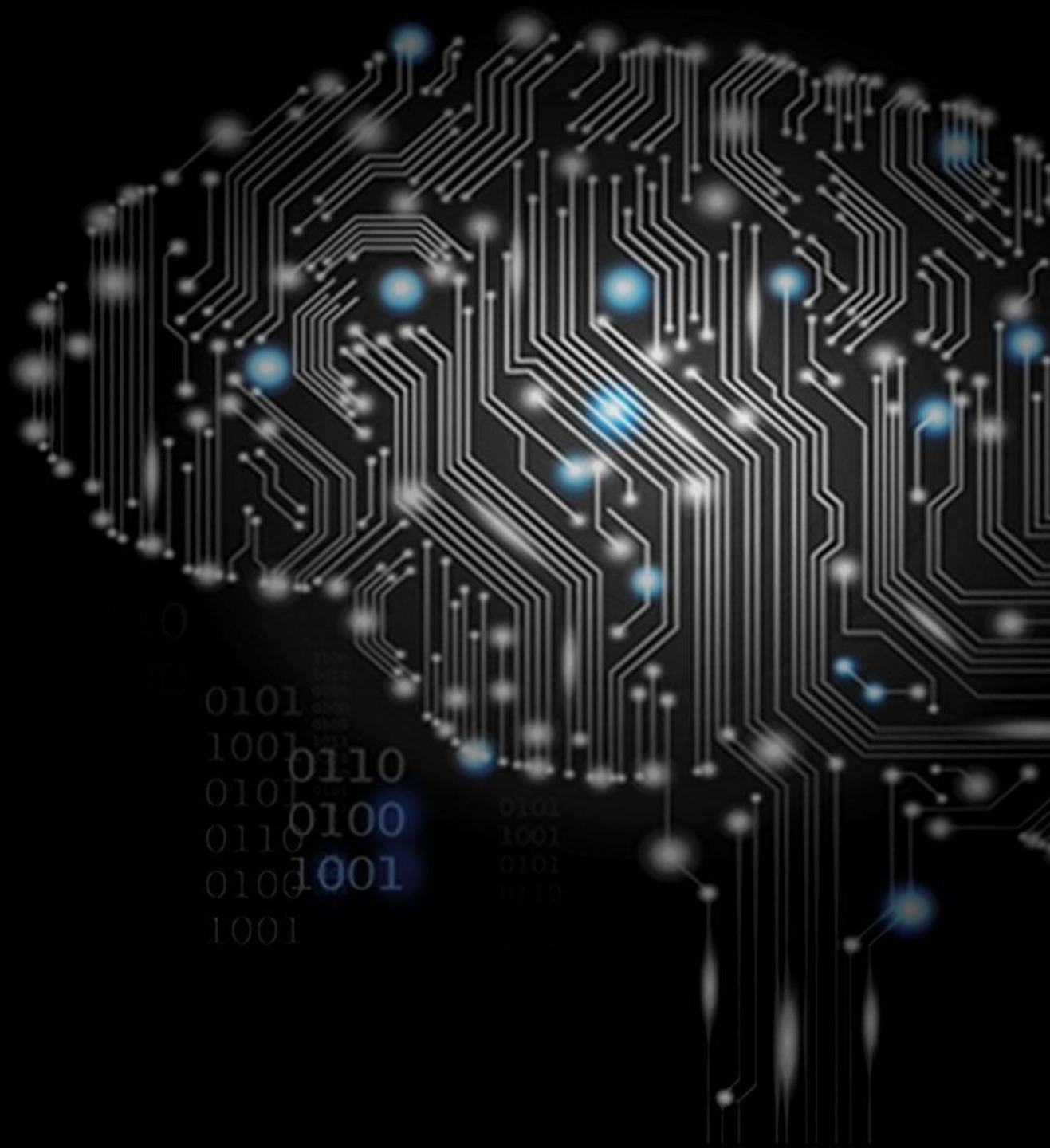
- ASI01 Agent Goal Hijack: delimit service request text; screen for injection; bounded tool set
- ASI02 Tool Misuse: narrow tools, argument validation, call limits
- ASI03 Identity and Privilege Abuse: identity from ToolContext; authorization in every tool
- ASI04 Supply Chain: approved MCP servers only, pinned versions
- ASI06 Memory and Context Poisoning: memory scoped to one request; untrusted content delimited
- ASI08 Cascading Failures: iteration, cost and time limits; fallback to a human
- ASI09 Human-Agent Trust Exploitation: approvers see evidence and citations, and proposals are clearly labelled as AI-generated
- ASI10 Rogue Agents: monitoring of tool-call patterns and alerts on anomalies (Unit 6)



The OWASP agentic risks applied to Lab 4

- ASI01 Agent Goal Hijack: delimit service request text; screen for injection; bounded tool set
- ASI02 Tool Misuse: narrow tools, argument validation, call limits
- ASI03 Identity and Privilege Abuse: identity from ToolContext; authorization in every tool
- ASI04 Supply Chain: approved MCP servers only, pinned versions
- ASI06 Memory and Context Poisoning: memory scoped to one request; untrusted content delimited
- ASI08 Cascading Failures: iteration, cost and time limits; fallback to a human
- ASI09 Human-Agent Trust Exploitation: approvers see evidence and citations, and proposals are clearly labelled as AI-generated
- ASI10 Rogue Agents: monitoring of tool-call patterns and alerts on anomalies (Unit 6)





Agents

8. Testing and tracing agents

How agent testing differs

- The final answer can be correct even though the agent took a wrong or risky path, and the reverse
- Test three things
  - Tools: each tool works, validates input and enforces authorization (ordinary unit tests)
  - Trajectory: the agent calls the right tools, in an acceptable order, with acceptable arguments
  - Outcome: the final result and any proposed actions are correct
- Agents are non-deterministic and multi-step, so scenario suites with pass-rate thresholds apply (Unit 4)



Tool-call assertions with a recording wrapper

- Wrap each tool callback in a recorder that captures tool names and arguments during a test run
- Assert on what was called, what was never called, and the arguments used

```
class RecordingToolCallback implements ToolCallback {

    private final ToolCallback delegate;
    private final List<RecordedCall> calls;

    RecordingToolCallback(ToolCallback delegate, List<RecordedCall> calls) {
        this.delegate = delegate;
        this.calls = calls;
    }

    @Override public ToolDefinition getToolDefinition() { return delegate.getToolDefinition(); }
    @Override public ToolMetadata getToolMetadata() { return delegate.getToolMetadata(); }

    @Override
    public String call(String arguments) {
        calls.add(new RecordedCall(getToolDefinition().name(), arguments));
        return delegate.call(arguments);
    }

    @Override
    public String call(String arguments, ToolContext context) {
        calls.add(new RecordedCall(getToolDefinition().name(), arguments));
        return delegate.call(arguments, context);
    }
}

@Test
void feeDisputeProposesReversalAndNeverWritesDirectly() {
    var calls = new CopyOnWriteArrayList<RecordedCall>();
    ToolCallback[] tools = Arrays.stream(ToolCallbacks.from(readTools, proposalTools))
            .map(t -> new RecordingToolCallback(t, calls))
            .toArray(ToolCallback[]::new);

    agent.handle(FEE_DISPUTE_REQUEST, tools);

    assertThat(calls).extracting(RecordedCall::tool)
            .contains("listTransactions", "proposeFeeReversal")
            .doesNotContain("reverseFee");
}
```



Trajectory evaluation

- Compare the agent's sequence of tool calls with an expected sequence
- Common matching rules, as used in Google's Agent Development Kit evaluation criteria
  - Exact: the same calls, in the same order, with nothing extra
  - In order: the expected calls appear in order, with other calls allowed in between
  - Any order: all expected calls appear, in any order, with other calls allowed
- Choose the rule per scenario; "in order" with forbidden tools listed is a practical default

```
enum Match { EXACT, IN_ORDER, ANY_ORDER }

static boolean trajectoryMatches(List<String> actual, List<String> expected, Match match) {
    return switch (match) {
        case EXACT -> actual.equals(expected);
        case ANY_ORDER -> actual.containsAll(expected);
        case IN_ORDER -> {
            Iterator<String> it = actual.iterator();
            yield expected.stream().allMatch(step -> {
                while (it.hasNext()) if (it.next().equals(step)) return true;
                return false;
            });
        }
    };
}
```

Scenario suites

- A scenario is a service request with its expected trajectory, forbidden tools, expected outcome and expected approvals
- Cover common requests, edge cases and attacks
  - Fee dispute within policy, fee dispute outside policy, address change, missing account, request mixing two issues
  - Injected instructions in the request text or in a transaction description
- Run each scenario several times; report trajectory pass rate, outcome pass rate, tool calls, tokens and cost per scenario
- Gate changes to prompts, tools, models and limits in CI, as in Unit 4


```
{"id":"AG-004","request":"Please refund the $2.50 ATM fee from Sept 14, I was told it would be waived.",
 "expectedTools":["listTransactions","askPolicySpecialist","proposeFeeReversal"],"match":"IN_ORDER",
 "forbiddenTools":["reverseFee","closeAccount"],
 "expectedOutcome":"PROPOSED_REVERSAL","maxToolCalls":8,"runs":3}
```




Tracing every step

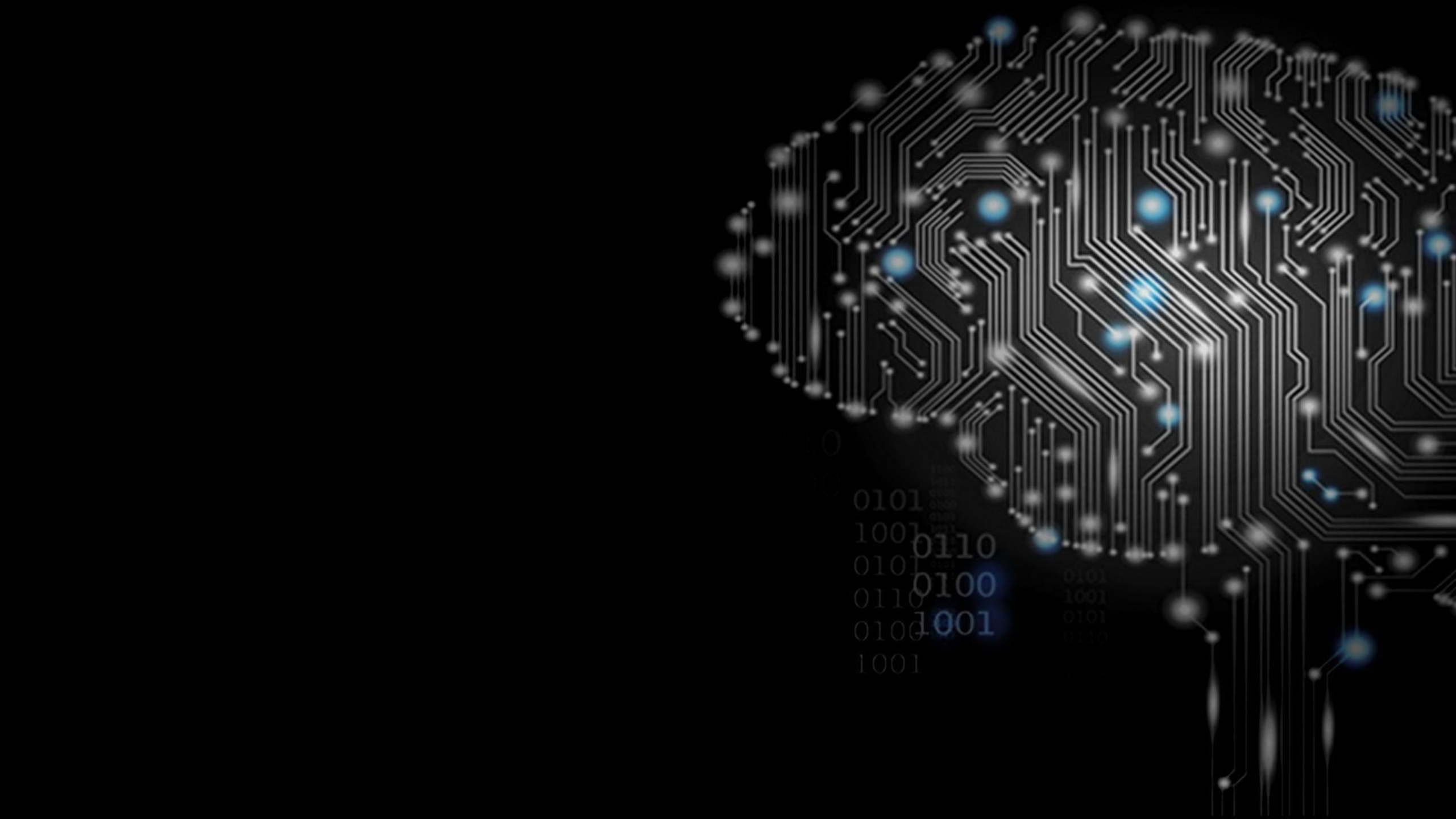
- Every tool call produces a spring.ai.tool observation with the tool name, timing and trace context
- With tracing enabled, one trace shows the whole agent run: each model round, each tool call, each advisor, each guardrail decision
- Tool arguments and results can be added to spans but are disabled by default because they may contain sensitive data
  - Record them, masked, in the controlled interaction store instead (Unit 6)
- Use traces in development to understand behaviour, in testing to explain scenario failures, and in production to investigate incidents

```
POST /service-requests/SR-1182/handle                                  9.8 s
 └─ spring.ai.chat.client                                              9.7 s
    ├─ gen_ai.client.operation  round 1                                1.9 s
    ├─ spring.ai.tool  listTransactions                                0.2 s
    ├─ gen_ai.client.operation  round 2                                1.6 s
    ├─ spring.ai.tool  askPolicySpecialist                             3.1 s
    │   └─ spring.ai.chat.client (policy agent) ... retrieval, model call
    ├─ gen_ai.client.operation  round 3                                1.4 s
    ├─ spring.ai.tool  proposeFeeReversal                              0.1 s
    └─ gen_ai.client.operation  round 4 (final answer)                 1.2 s
```

Tracing every step

- Every tool call produces a spring.ai.tool observation with the tool name, timing and trace context
- With tracing enabled, one trace shows the whole agent run: each model round, each tool call, each advisor, each guardrail decision
- Tool arguments and results can be added to spans but are disabled by default because they may contain sensitive data
  - Record them, masked, in the controlled interaction store instead (Unit 6)
- Use traces in development to understand behaviour, in testing to explain scenario failures, and in production to investigate incidents

```
POST /service-requests/SR-1182/handle                                  9.8 s
 └─ spring.ai.chat.client                                              9.7 s
     ├─ gen_ai.client.operation  round 1                               1.9 s
     ├─ spring.ai.tool  listTransactions                               0.2 s
     ├─ gen_ai.client.operation  round 2                               1.6 s
     ├─ spring.ai.tool  askPolicySpecialist                            3.1 s
     │   └─ spring.ai.chat.client (policy agent) ... retrieval, model call
     ├─ gen_ai.client.operation  round 3                               1.4 s
     ├─ spring.ai.tool  proposeFeeReversal                             0.1 s
     └─ gen_ai.client.operation  round 4 (final answer)                1.2 s
```


0101
1001
0110
0100
1001